Splunk SIEM Log Monitoring & Detection Workflow

In this lab, I built a Splunk-based SIEM detection pipeline that mirrors the workflow implemented in Lab 5. The purpose of this lab is to demonstrate that the same attack scenarios and detection logic can be reproduced using a different SIEM platform.

While Lab 5 focused on Wazuh, this lab replaced the SIEM with Splunk Enterprise. Network-based alerts generated by Suricata and host-based SSH authentication logs are ingested, analyzed, and correlated to reconstruct attacker behavior.

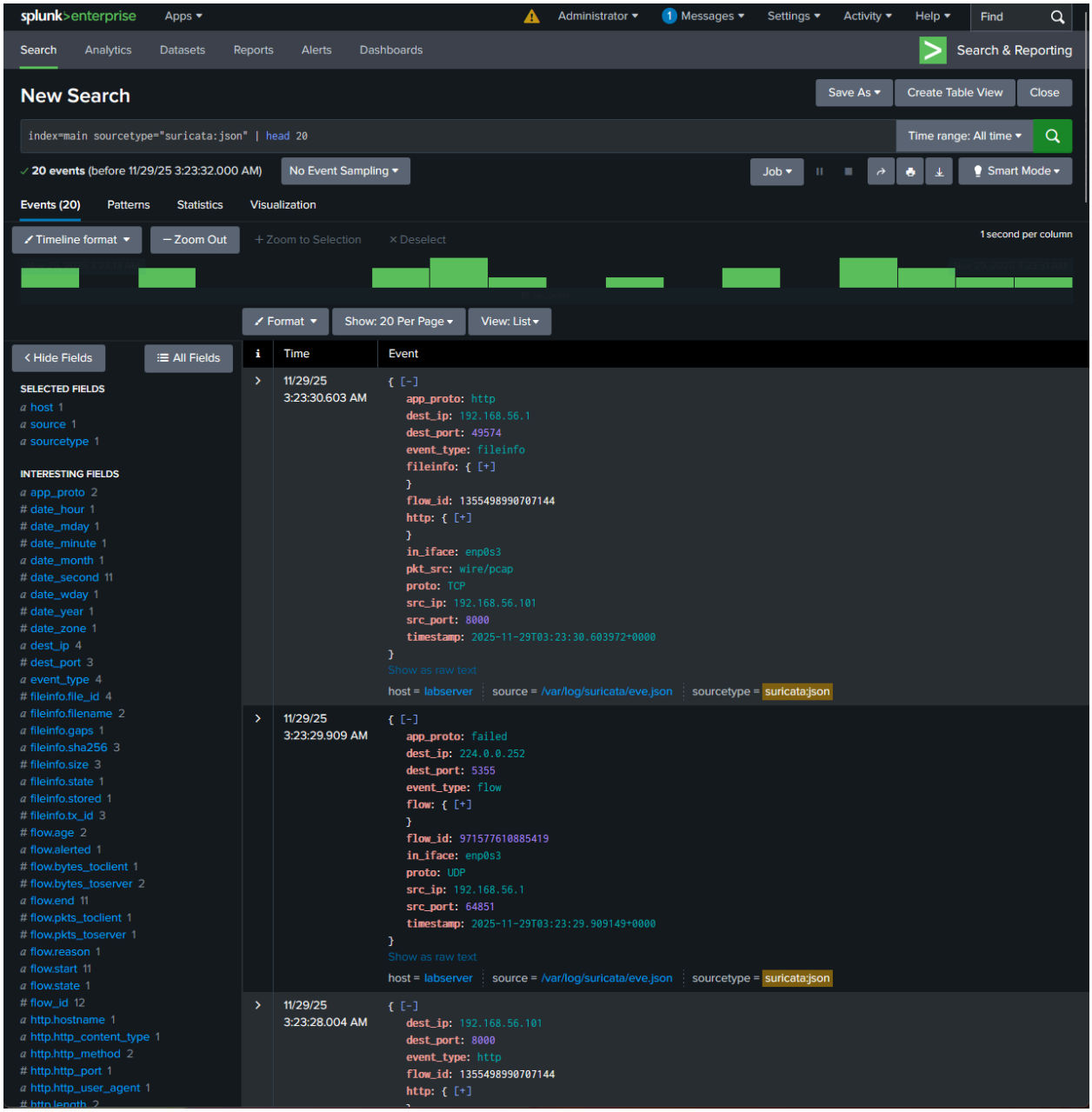
Nmap and Hydra attacks are launched from a Kali Linux attacker to simulate reconnaissance and SSH brute-force activity.

The lab environment consists of two virtual machines connected through a VirtualBox host-only network. Kali Linux is used as the attacker system, while Ubuntu Server acts as the target machine and hosts both Suricata and Splunk Enterprise.

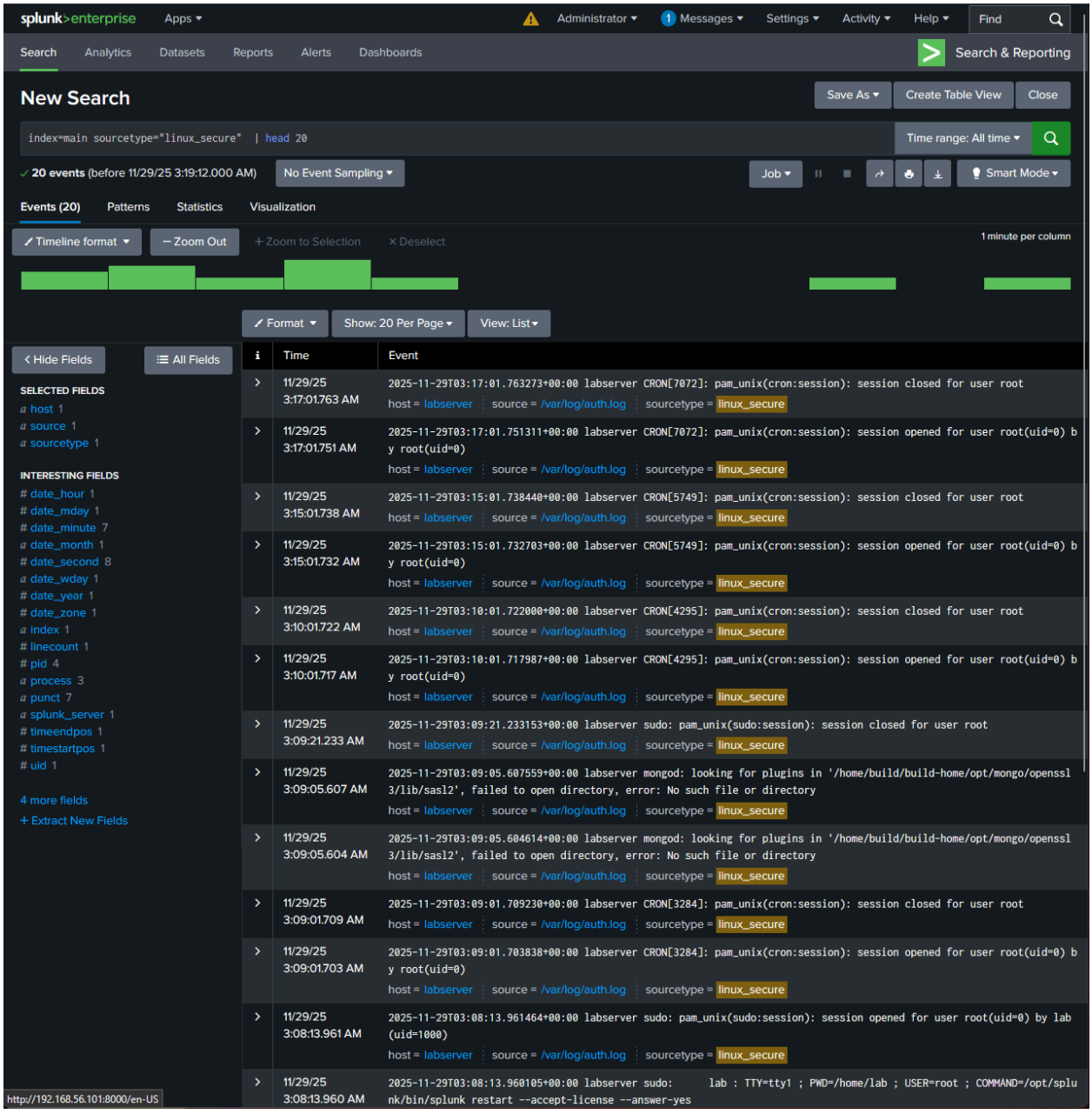
Suricata inspects network traffic and writes events to a JSON log file, while SSH authentication activity is recorded in the system authentication logs. Both log sources are ingested into Splunk under a single index to enable centralized analysis.

I first verified that Suricata alerts were successfully ingested into Splunk. The Suricata `eve.json` file was monitored and indexed using the `suricata:json` sourcetype.

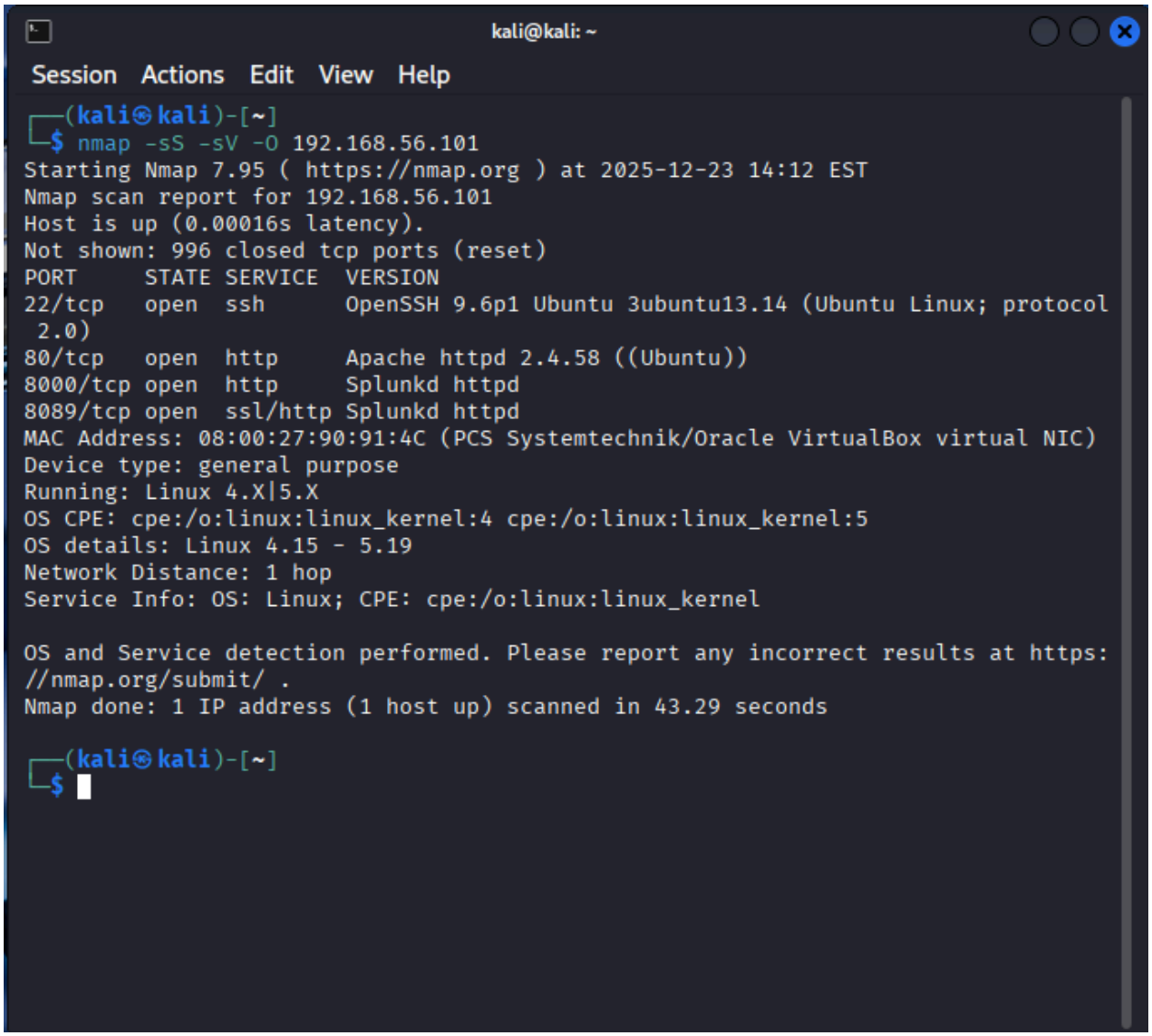
After ingestion, I confirmed that JSON-formatted Suricata events were searchable in Splunk, indicating that network traffic data was being collected correctly.



Next, I verified ingestion of Linux SSH authentication logs from `/var/log/auth.log`. These logs were indexed using the `linux_secure` sourcetype. SSH login attempts, sudo activity, and other authentication-related events were visible in Splunk, confirming successful host-based log ingestion.

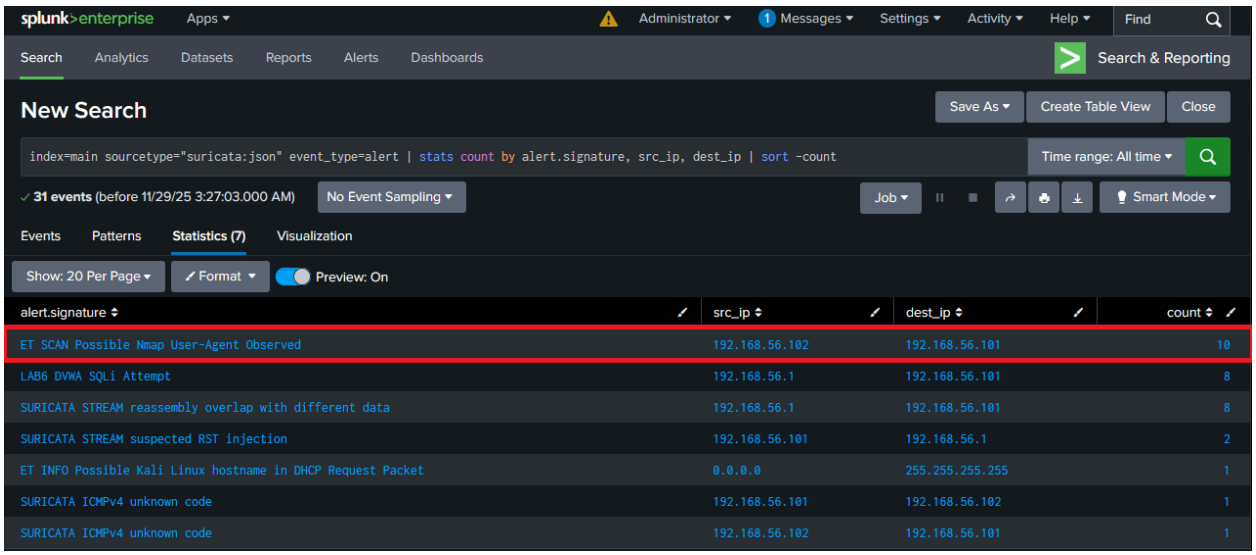


To simulate reconnaissance activity, I launched an Nmap scan from the Kali Linux attacker against the Ubuntu server. The scan included SYN scanning and service detection to enumerate open ports and running services. This attack represents the initial reconnaissance phase commonly observed in real-world intrusion attempts.



After executing the Nmap scan, I analyzed Suricata alerts in Splunk to identify scan-related detections.Suricata generated alerts such as *ET SCAN Possible Nmap User-Agent Observed*,

clearly indicating network reconnaissance activity originating from the attacker IP. The source and destination IP addresses confirmed the attack path.

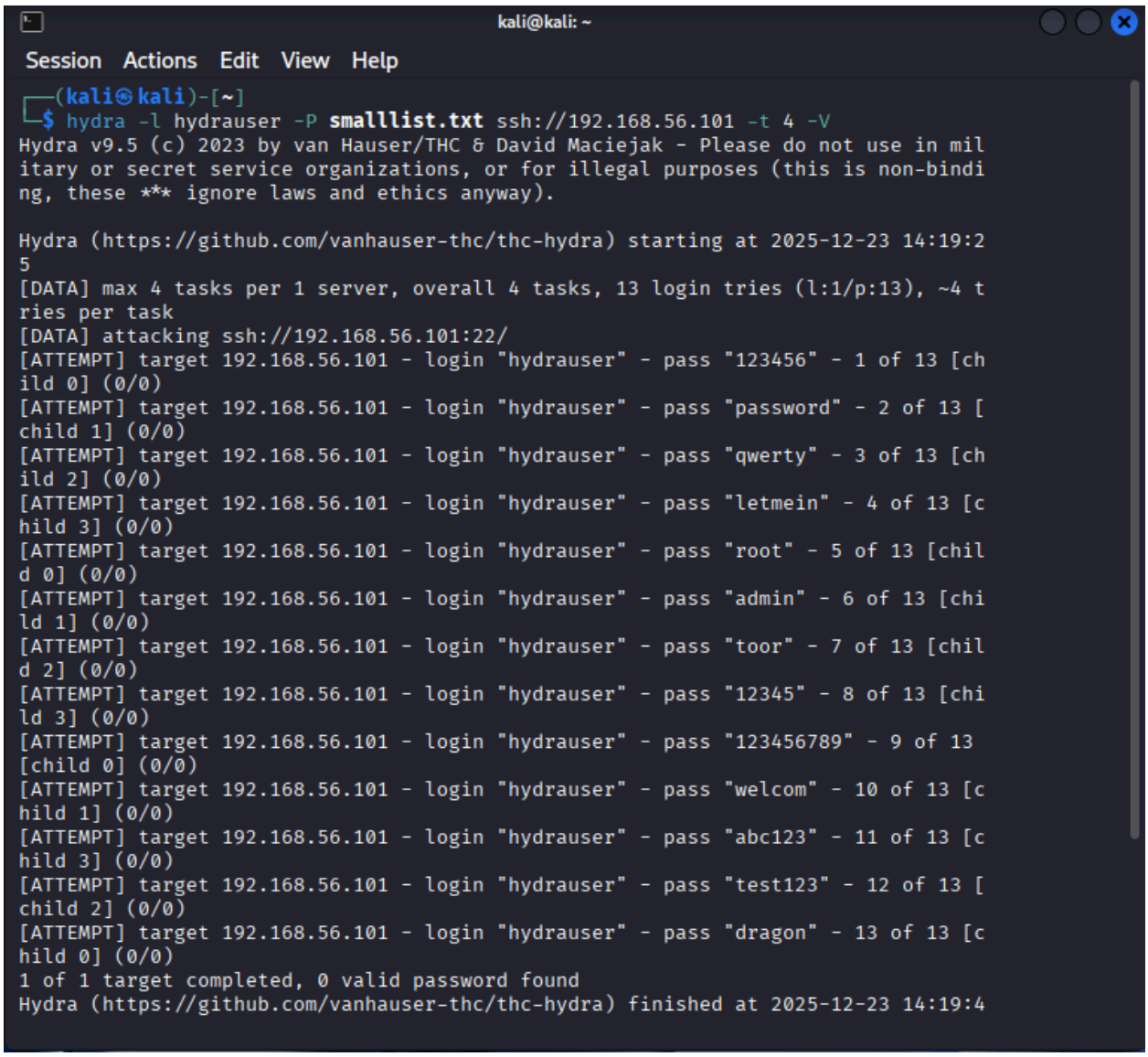


To safely test SSH brute-force behavior, I created a dedicated test account on the Ubuntu server. This ensured that the attack did not impact legitimate user accounts.

```
lab@labserver:~$ sudo adduser hydrauser
[sudo] password for lab:
info: Adding user `hydrauser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hydrauser' (1002) ...
info: Adding new user `hydrauser' (1002) with group `hydrauser (1002)' ...
info: Creating home directory `/home/hydrauser' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hydrauser
Enter the new value, or press ENTER for the default
        Full Name []: hydrauser
        Room Number []: 203
        Work Phone []: 1112223333
        Home Phone []: 1112223333
        Other []:
Is the information correct? [Y/n] y
info: Adding new user `hydrauser' to supplemental / extra groups `users' ...
info: Adding user `hydrauser' to group `users' ...
lab@labserver:~$
```

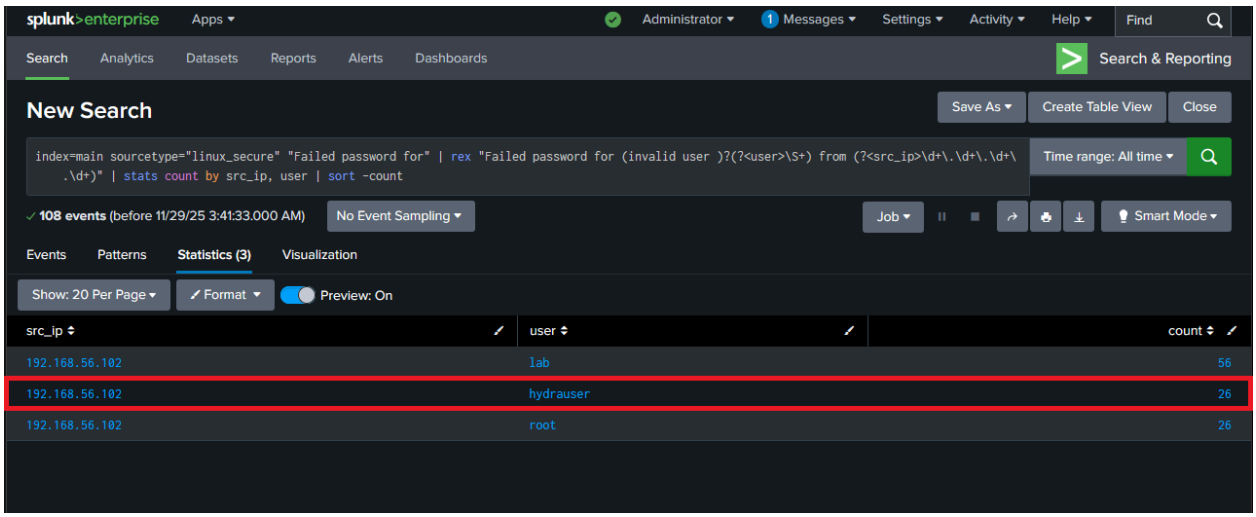
Using the Kali attacker, I executed a Hydra attack against the SSH service on the Ubuntu server. Hydra attempted multiple password guesses for the test account, generating repeated

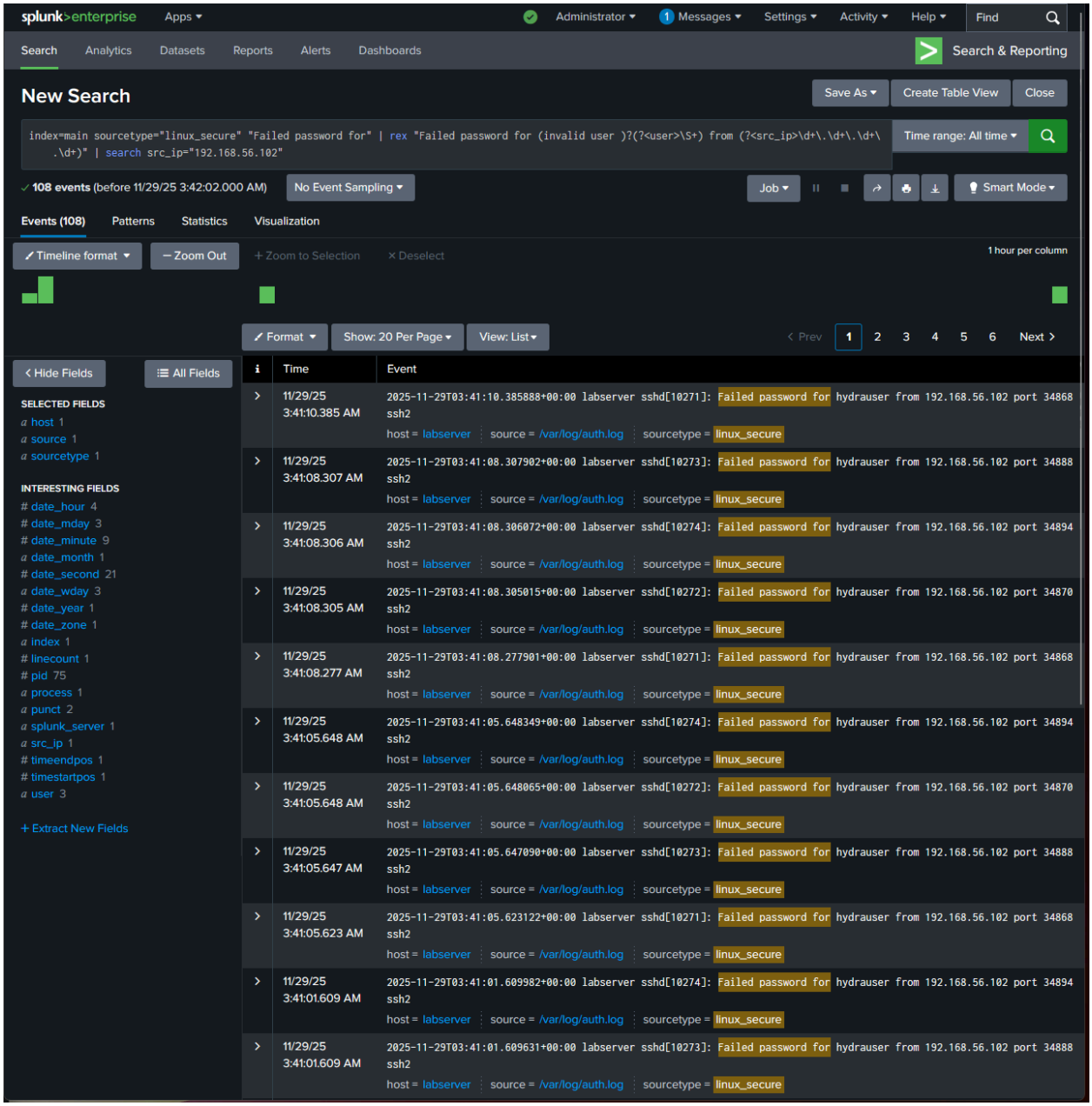
authentication failures. This simulates credential brute-force behavior commonly associated with unauthorized access attempts.



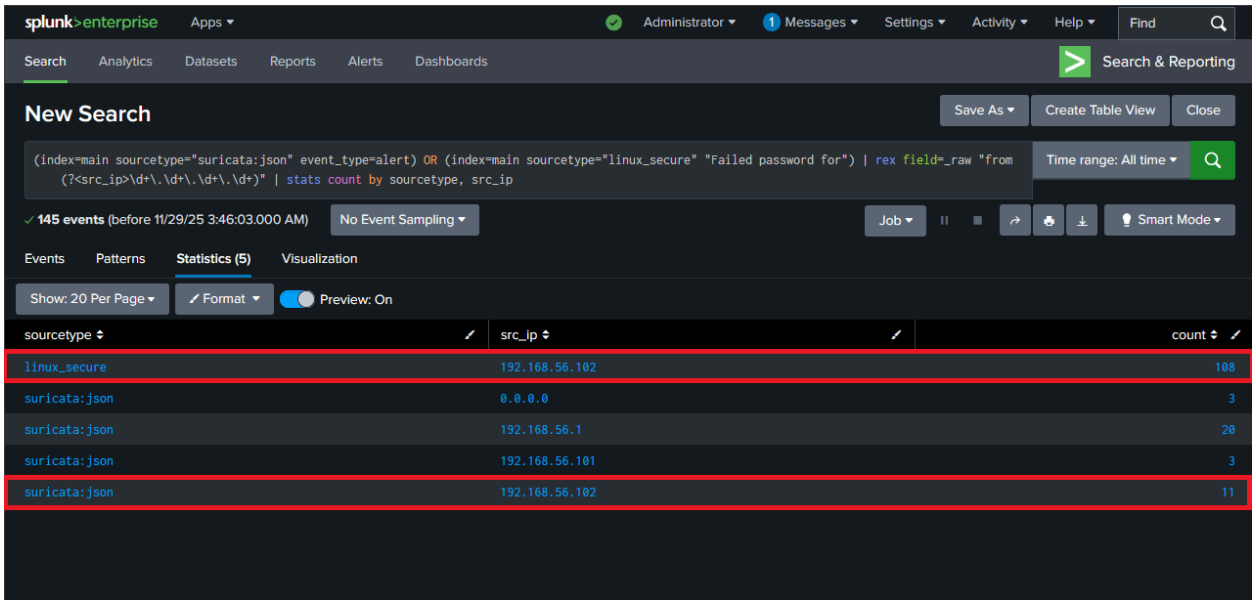
I analyzed SSH authentication logs in Splunk to detect brute-force activity. Failed password attempts were identified using the `linux_secure` sourcetype. Since SSH logs do not natively provide a structured source IP field, I used regular expression extraction to parse the attacker IP

and username from raw log messages. The results showed a high number of failed login attempts originating from the Kali attacker IP, confirming brute-force behavior.




splunk>enterprise
Apps
Administrator
1 Messages
Settings
Activity
Help
Find
Search
Analytics
Datasets
Reports
Alerts
Dashboards
Search & Reporting
New Search
Save As
Create Table View
Close
index=main sourcetype="linux_secure" "Failed password for" | rex "Failed password for (invalid user)?(?<user>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | search src_ip="192.168.56.102"
Time range: All time
108 events (before 11/29/25 3:42:02.000 AM)
No Event Sampling
Job
Smart Mode
Events (108)
Patterns
Statistics
Visualization
Timeline format
Zoom Out
Zoom to Selection
Deselect
1 hour per column
Format
Show: 20 Per Page
View: List
Prev
1
2
3
4
5
6
Next
Hide Fields
All Fields
SELECTED FIELDS
a host 1
a source 1
a sourcetype 1
INTERESTING FIELDS
date_hour 4
date_mday 3
date_minute 9
a date_month 1
date_second 21
a date_wday 3
date_year 1
date_zone 1
a index 1
linecount 1
pid 75
a process 1
a punct 2
a splunk_server 1
a src_ip 1
timeendpos 1
timestartpos 1
a user 3
Extract New Fields
Time
Event
11/29/25 3:41:10.385 AM
2025-11-29T03:41:10.385888+00:00 labserver sshd[10271]: Failed password for hydrauser from 192.168.56.102 port 34868 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:08.307 AM
2025-11-29T03:41:08.307902+00:00 labserver sshd[10273]: Failed password for hydrauser from 192.168.56.102 port 34888 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:08.306 AM
2025-11-29T03:41:08.306072+00:00 labserver sshd[10274]: Failed password for hydrauser from 192.168.56.102 port 34894 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:08.305 AM
2025-11-29T03:41:08.305015+00:00 labserver sshd[10272]: Failed password for hydrauser from 192.168.56.102 port 34870 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:08.277 AM
2025-11-29T03:41:08.277901+00:00 labserver sshd[10271]: Failed password for hydrauser from 192.168.56.102 port 34868 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:05.648 AM
2025-11-29T03:41:05.648349+00:00 labserver sshd[10274]: Failed password for hydrauser from 192.168.56.102 port 34894 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:05.648 AM
2025-11-29T03:41:05.648065+00:00 labserver sshd[10272]: Failed password for hydrauser from 192.168.56.102 port 34870 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:05.647 AM
2025-11-29T03:41:05.647090+00:00 labserver sshd[10273]: Failed password for hydrauser from 192.168.56.102 port 34888 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:05.623 AM
2025-11-29T03:41:05.623122+00:00 labserver sshd[10271]: Failed password for hydrauser from 192.168.56.102 port 34868 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:01.609 AM
2025-11-29T03:41:01.609982+00:00 labserver sshd[10274]: Failed password for hydrauser from 192.168.56.102 port 34894 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure
11/29/25 3:41:01.609 AM
2025-11-29T03:41:01.609631+00:00 labserver sshd[10273]: Failed password for hydrauser from 192.168.56.102 port 34888 ssh2
host = labserver source = /var/log/auth.log sourcetype = linux_secure

To mirror the correlation logic used in Lab 5, I correlated Suricata alerts and SSH authentication failures by attacker IP. By normalizing the source IP field across both network and host-based logs, I confirmed that the same attacker IP was responsible for both reconnaissance activity and SSH brute-force attempts. This correlation reconstructs a multi-stage attack sequence originating from a single source.



After validating individual detections, I created a Splunk dashboard to visualize the full detection workflow. The dashboard includes panels summarizing Suricata alerts, Nmap scan activity, SSH brute-force attempts, and correlation by attacker IP. Its structure closely mirrors the one used in Lab 5, with Splunk searches replacing Wazuh rules.

The results show a clear attack progression from reconnaissance to attempted access. Network scans were detected by Suricata, while repeated SSH authentication failures were recorded at the host level. Correlating these logs provides a coherent view of attacker behavior across multiple data sources.

This lab relies on manual SPL searches rather than automated alerts. Additionally, the environment represents a small-scale lab and does not reflect enterprise-level traffic volume or complexity. Also, it demonstrates that SIEM detection workflows are portable across platforms when designed correctly. By reproducing the same attack scenarios and detection logic used in Lab 5, I confirmed that Splunk can effectively ingest, detect, correlate, and visualize security events using both network and host-based logs.